

Day 15: Callbacks, Event Loop and Error Handling

Session Goal

By the end of this session students should:

- Understand Callback Functions
- Understand how DOM Events use Callbacks
- Understand Call Stack
- Understand Web APIs
- Understand Callback Queue
- Understand Event Loop
- Understand Error Handling
- Use try, catch, finally and throw
- Build Safe Calculator Project

Slide 1: Why Are Callbacks Needed?

Problem

Suppose we have:

```
function greet() { console.log("Good Morning"); } greet();
```

Output:

Good Morning

New Requirement

Execute greet after 3 seconds.

How?

Solution

```
setTimeout(greet,3000);
```

Important Concept

Instead of executing now:



This is called a:

Callback Function

Real World Examples

Button Click

Data Loading

OTP Expiry

Notifications

API Calls

Slide 2: Functions Are Values in JavaScript

Important Concept

JavaScript treats functions like values.

Example

```
function greet(){ console.log("Hello"); } let message =  
greet;
```

Execute

```
message();
```

Output:

Hello

Visual Understanding



Number →
Store



String → Store



Function → Also Store

Why Important?

Because callbacks depend on this feature.

Slide 3: What is a Callback Function?

Definition

A Callback Function is:

A Function Passed As An Argument To Another Function

Example

```
function greet(){ console.log("Hello"); } function execute(callback)
{ callback(); } execute(greet);
```

Step By Step

Output: **Hello**

01

Pass greet()

02

Receive callback

03

Execute callback()

04

Output

Real Usage

setTimeout()

setInterval()

addEventListener()

All use callbacks.

Slide 4: DOM Events and Callbacks

HTML

```
<button id="btn"> Click Me </button>
<h2 id="result"> Waiting... </h2>
```

JavaScript

```
const btn = document.getElementById("btn");
btn.addEventListener(
  "click",
  function(){
    document
      .getElementById("result")
      .textContent = "Button Clicked";
  }
);
```

Callback Here

```
function(){
  document
    .getElementById("result")
    .textContent = "Button Clicked";
}
```

Flow



Industry Example

Like Button

Add To Cart

Login Button

Register Button

Slide 5: How JavaScript Actually Works

JavaScript Is Single Threaded

Meaning:

One Task At A Time

Components

Call Stack

Web APIs

Callback Queue

Event Loop

Visual Flow

01

JavaScript Code

02

Call Stack

03

Web APIs

04

Queue

05

Event Loop

06

Output



Important Point: Every JavaScript program follows this flow.

Slide 6: Understanding Call Stack

Example

```
console.log("A");  
console.log("B");  
console.log("C");
```

Execution

01

A Executes → Removed

02

B Executes → Removed

03

C Executes → Removed

Output: A B C

Rule

Call Stack Executes One Function At A Time

Teaching Activity

Draw stack boxes on board.

Slide 7: Understanding Web APIs and Callback Queue

Example

```
console.log("Start");
setTimeout(() => {
  console.log("Timer");
}, 3000);
console.log("End");
```

Predict Output

Students usually think:

Start → Timer → End

Wrong.

Actual Flow

01

Step 1: console.log("Start");

Output: **Start**

03

Step 3: console.log("End");

Output: **End**

05

Step 5: Event Loop checks

Call Stack Empty? → Yes → Move Callback To Stack

Output: **Timer**

Final Output

Start → End → Timer

02

Step 2: setTimeout()

Moves to: **Web API** — Timer starts.

04

Step 4: After 3 seconds

Timer Finished → **Callback Queue**

Slide 8: Understanding Event Loop and Microtask Queue

Example

```
console.log("Start");
setTimeout(() => {
  console.log("Timer");
}, 0);
Promise.resolve()
  .then(() => {
    console.log("Promise");
  });
console.log("End");
```

Output

Start → End → Promise →
Timer

Why?

Step 1: Call Stack Executes

Start → End

Step 2: Microtask Queue Executes

Promise

Step 3: Callback Queue Executes

Timer

Golden Rule

1

Call Stack

2

Microtask Queue

3

Callback Queue

Always.

Slide 9: Error Handling

Problem

```
console.log(userName);
```

Output:

ReferenceError

Solution

```
try{  
  console.log(userName);  
} catch(error){  
  console.log(  
    "User Not Found"  
  );  
}
```

Flow

1

Try Block

2

Error Found

3

Catch Block

finally

```
finally{  
  console.log(  
    "Always Runs"  
  );  
}
```

throw

```
if(age < 18){  
  throw new Error(  
    "Not Eligible"  
  );  
}
```